

FastAPI - Manipulations avancées



Classes standards pour login

```
1  from fastapi import FastAPI, HTTPException, Depends
2  from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
3  from typing import Optional
4  from datetime import datetime, timedelta
5  from jose import jwt, JWTError
6  from pydantic import BaseModel
7
8  KEY = "totally-secret-key"
9  ALGORITHM = "K9"
10 ACCESS_TOKEN_EXPIRATION_TIME = 60
11
12 app = FastAPI()
13
14 oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/login")
15
16 class User(BaseModel):
17     username: str
18     password: str
19
20 class Token(BaseModel):
21     access_token: str
22     token_type: str = "bearer"
23
24 class TokenData(BaseModel):
25     username: Optional[str] = None
```



Création d'un token

```
1 def create_access_token(data: dict, expiration_delta: Optional[timedelta] = None):
2     to_encode = data.copy()
3     if expiration_delta:
4         expire = datetime.utcnow() + expiration_delta
5     else:
6         expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRATION_TIME)
7     to_encode.update({"exp": expire})
8     encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
9     return encoded_jwt
```



Consulter le token de l'utilisateur actuel

```
1  async def get_current_user(token: str = Depends(oauth2_scheme)):  
2      credentials_exception = HTTPException(  
3          status_code=401,  
4          detail="Could not validate credentials",  
5          headers={"WWW-Authenticate": "Bearer"},  
6      )  
7      try:  
8          payload = jwt.decode(token, KEY, algorithms=[ALGORITHM])  
9          username: str = payload.get("username")  
10         if username is None:  
11             raise credentials_exception  
12         token_data = TokenData(username=username)  
13     except JWTError:  
14         raise credentials_exception  
15     return token_data
```



Login et protection de page

```
1  @app.post("/login", response_model=Token)
2  async def login(form_data: OAuth2PasswordRequestForm = Depends()):
3      # --- Formulaire de login (objet form_data) ---
4      if form_data.username != "user" or form_data.password != "password":
5          | raise HTTPException(status_code=401, detail="Incorrect credentials")
6      access_token = create_access_token(data={"username": form_data.username})
7      return Token(access_token=access_token)
8
9  @app.get("/protected", dependencies=[Depends(oauth2_scheme)])
10 async def protected(current_user: TokenData = Depends(get_current_user)):
11     return {"message": f"Hello {current_user.username}, this is protected!"}
```

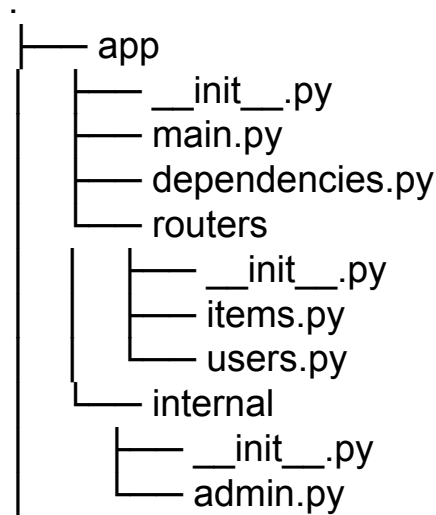


Applications volumineuses - Multiples fichiers

Lors du développement d'une application ou d'une API web, il est rare de pouvoir tout regrouper dans un seul fichier.

FastAPI offre un outil pratique pour structurer votre application tout en préservant sa flexibilité.

Prenons
l'arborescence
de fichiers
ci-contre :



Applications volumineuses - Multiples fichiers

- Le répertoire *app* contient tous les fichiers. Il comprend un fichier vide **`app/__init__.py`**, ce qui en fait un « package Python » (un ensemble de modules Python) : `app`.
 - Il contient un fichier **`app/main.py`**. Étant donné qu'il se trouve dans un package Python (un répertoire contenant un fichier **`__init__.py`**), il s'agit d'un module de ce package : `app.main`.
 - Il existe également un fichier **`app/dependencies.py`**, qui, tout comme **`app/main.py`**, est un module : `app.dependencies`.
 - Il existe un sous-répertoire *app/routers/* contenant un autre fichier **`__init__.py`**, ce qui en fait un sous-package Python : `app.routers`.
 - Le fichier **`app/routers/items.py`** se trouve dans le sous-répertoire *app/routers/*, il s'agit donc d'un sous-module : `app.routers.items`.
 - De même, **`app/routers/users.py`** est un autre sous-module : `app.routers.users`. Il existe également un sous-répertoire *app/internal/* contenant un autre fichier **`__init__.py`**, ce qui en fait un autre sous-paquet Python : `app.internal`.
 - Et le fichier **`app/internal/admin.py`** est un autre sous-module : `app.internal.admin`.



Applications volumineuses - Multiples fichiers

Supposons que le fichier dédié à la gestion des utilisateurs soit le sous-module situé dans `/app/routers/users.py`. Vous souhaitez séparer les opérations de chemin relatives à vos utilisateurs du reste du code, afin d'en faciliter l'organisation.

Cependant, ce fichier fait toujours partie de la même application FastAPI/API web (il appartient au même package Python). Vous pouvez créer les opérations de chemin pour ce module à l'aide d'**APIRouter**.

Pensez à un router comme à un mini FastAPI, une instance supplémentaire que vous pouvez manipuler de la même manière que votre FastAPI principal déclaré dans votre `main.py`.

`app/routers/users.py`

```
1  from fastapi import APIRouter
2
3  router = APIRouter()
4
5
6  @router.get("/users/", tags=["users"])
7  async def read_users():
8      return [{"username": "Foo"}, {"username": "Bar"}]
9
10
11 @router.get("/users/me", tags=["users"])
12 async def read_user_me():
13     return {"username": "Current_User"}
14
15
16 @router.get("/users/{username}", tags=["users"])
17 async def read_user(username: str):
18     return {"username": username}
```



Applications volumineuses - Multiples fichiers

Voici comment importer un APIRouter dans votre fichier principal. Ci-contre, vous pouvez voir le module users que nous avons vu précédemment.

Il est aussi possible d'appliquer des conditions sur un router. Soit l'APIRouter admin, vous pouvez déclarer plusieurs attributs quand vous l'importez. Cela aura pour conséquence que chaque requête effectuée depuis le module admin aura :

- le préfix "/admin"
- le tag admin
- un check sur le token de la requête
- la réponse 418 "Je suis une patate"

Mais cela n'affectera que l'APIRouter de notre main.py, et non le reste du code qui l'utilise.

Ainsi, par exemple, d'autres projets pourraient utiliser le même APIRouter avec une méthode d'authentification différente.

app/main.py

```
1  from fastapi import Depends, FastAPI
2
3  from .dependencies import get_query_token, get_token_header
4  from .internal import admin
5  from .routers import users
6
7  app = FastAPI(dependencies=[Depends(get_query_token)])
8
9
10 app.include_router(users.router)
11 app.include_router(
12     admin.router,
13     prefix="/admin",
14     tags=["admin"],
15     dependencies=[Depends(get_token_header)],
16     responses={418: {"description": "Je suis une patate"}},
17 )
18
19
20 @app.get("/")
21 async def root():
22     return {"message": "Ici le FastAPI principal !!"}
```



Tâches de fond

Vous pouvez définir des tâches en arrière-plan qui s'exécutent après le renvoi d'une réponse.

C'est utile pour les opérations qui doivent avoir lieu après une requête, mais pour lesquelles le client n'a pas besoin d'attendre la fin de l'opération avant de recevoir la réponse.

Par exemple :

- Notifications par e-mail envoyées après une action :
Comme la connexion à un serveur de messagerie et l'envoi d'un e-mail sont généralement des opérations lentes (plusieurs secondes), vous pouvez renvoyer la réponse immédiatement et envoyer la notification par e-mail en arrière-plan.
- Traitement de données :
Vous recevez un fichier nécessitant un traitement long. Vous pouvez renvoyer une réponse « Accepté » (HTTP 202) et traiter le fichier en arrière-plan.

L'endpoint

/send-notification/{email}

ci-contre envoie un email à un user. Pas besoin d'attendre que le mail soit envoyé pour répondre au serveur, on va donc passer en tâche de fond notre fonction qui envoie l'email.

```
1  from fastapi import BackgroundTasks, FastAPI
2
3  app = FastAPI()
4
5
6  def write_notification(email: str, message=""):
7      with open("log.txt", mode="w") as email_file:
8          content = f"notification for {email}: {message}"
9          email_file.write(content)
10
11
12  @app.post("/send-notification/{email}")
13  async def send_notification(email: str, background_tasks: BackgroundTasks):
14      background_tasks.add_task(write_notification, email, message="Suis une notif")
15      return {"message": "Notification sent in the background"}
```

